# CVE-2026-46300 - Fragnesia

Two points are worth establishing upfront. First, this is the same broad attack surface as Dirty Frag (XFRM, ESP, the page cache) but it lives in a different code path. The original Dirty Frag bugs were in `esp_input()` and `rxkad_verify_packet_1()`. Fragnesia is in `skb_try_coalesce()`, the function that merges socket buffers. Second, the published proof-of-concept covers only the espintcp variant; a second variant in `skb_segment()`

| Property | Copy Fail | Dirty Frag | Fragnesia |
|---|---|---|---|
| CVE | CVE-2026-31431 | CVE-2026-43284 / 43500 | CVE-2026-46300 |
| Kernel subsystem | AF_ALG / splice | xfrm-ESP UDP / RxRPC | xfrm-ESP-in-TCP / skb coalescing |
| Write primitive size | 4 bytes | 4 bytes (ESP) / 8 bytes (RxRPC) | 1 byte (per trigger) |
| Race condition required | No | No | No |
| Container escape impact | Yes | Yes | Yes |
| Patch state | Mainline | xfrm-ESP mainline; RxRPC unpatched | First variant on netdev; second variant unpatched |

# How a Patch Becomes a Vulnerability

### SKBFL_SHARED_FRAG: The Honesty Flag

A socket buffer ( `struct sk_buff` ) carries an array of fragments, each describing a memory page that belongs to the packet. Most fragments are private buffers owned by the network stack. Some, however, are references to pages that belong to other subsystems, most commonly the page cache. When a process calls `splice()` to send a file's contents through a socket, the kernel attaches a reference to the file's page-cache page directly to the outgoing skb's fragment list, without copying any bytes.

The `SKBFL_SHARED_FRAG` flag is the kernel's way of marking such fragments as externally owned. The flag tells every downstream code path one thing: "this memory does not belong to you; if you need to modify it, copy it first." Functions that need to mutate skb data check `skb_has_shared_frag()` and call `skb_cow_data()` to obtain a private buffer before doing anything destructive.

The flag is an invariant. As long as every code path that touches a shared fragment respects it, page-cache pages stay safe even as they pass through the network stack.

### skb_try_coalesce: Where the Invariant Breaks

`skb_try_coalesce()` is a kernel function that merges two skbs into one. It is used by the TCP stack on the receive side to combine queued segments into larger buffers, reducing memory overhead and improving throughput. When two skbs are coalesced, the second skb's fragments are transferred onto the first skb's fragment array, and the second skb is freed.

This is where the bug lives. When `skb_try_coalesce()` transfers paged fragments between buffers, it does not propagate the `SKBFL_SHARED_FRAG` marker on those fragments. The coalesced skb ends up carrying fragments that are still backed by the page cache, but with the flag cleared. To every downstream code path, the coalesced skb now looks like a perfectly ordinary private buffer that can be modified in place.

The bug was introduced in 2013 (commit `cef401de7be8` ) and sat dormant for thirteen years, because for most of that time no production code path looked at coalesced skbs and decided to write to them. The Dirty Frag patch changed that.

### The XFRM ESP-in-TCP Receive Path

ESP-in-TCP (RFC 8229) is an IPsec transport mode that encapsulates ESP packets inside a TCP stream. It is used in NAT-traversal scenarios where UDP-based ESP encapsulation does not work. When a TCP socket is configured for espintcp via `setsockopt` , the kernel switches the socket's Upper Layer Protocol (ULP) handler to one that pulls ESP packets out of the TCP byte stream and feeds them to `esp_input()` .

`esp_input()` is the function the original Dirty Frag bug lived in. It performs AEAD decryption on the incoming packet's payload, writing the plaintext back into the skb. For performance, the decryption is done in place when the input skb is uncloned and has no frag list. The function calls `skb_has_shared_frag()` to verify that none of the fragments are externally backed. If the flag is clear, the function takes the fast path; if it is set, the function falls back to allocating a private buffer via `skb_cow_data()` before decrypting.

This is exactly the check the Dirty Frag patch added. Before the patch, the function did not always honour the flag. After the patch, the function does. The patch was correct. The flag, however, is no longer trustworthy after `skb_try_coalesce()` strips it.

## The Combination That Becomes Fragnesia

1. Attacker opens a TCP socket and splices /usr/bin/su's page cache pages into the socket's receive queue (skbs are flagged SKBFL_SHARED_FRAG).
2. Kernel coalesces queued skbs via skb_try_coalesce(); the flag is lost.

3. Attacker calls setsockopt(TCP_ULP, "espintcp") to switch the socket to ESP-in-TCP mode. Queued data is now processed as ESP ciphertext.
4. esp_input() checks the (now-clear) shared-frag flag, takes the fast path, performs AES-GCM decryption in place over the page cache page.
5. The decryption is a XOR of the AES-GCM keystream against the ciphertext. By choosing the IV, the attacker chooses the keystream byte, which chooses the byte written into the page cache.

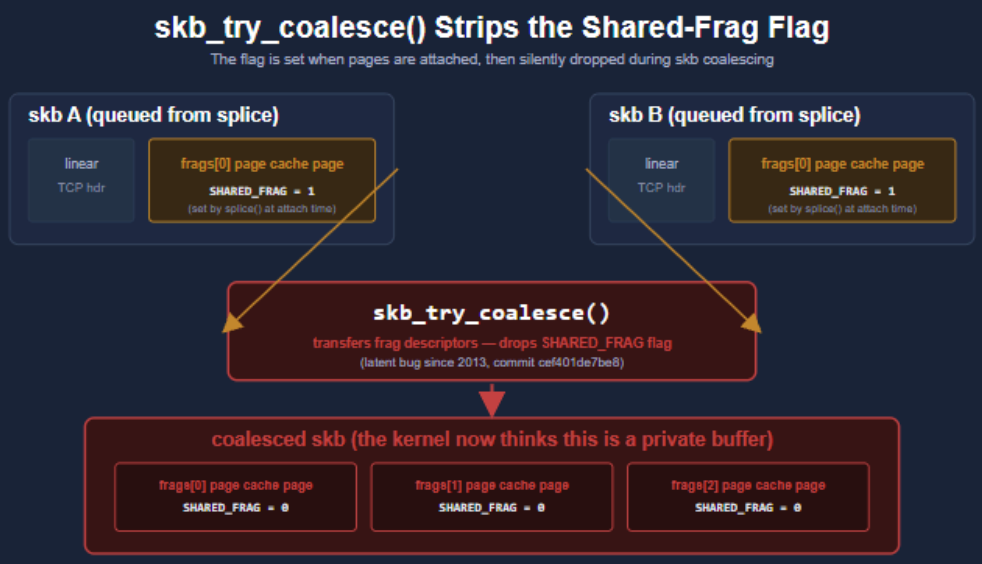
The technique is called Fragnesia (a portmanteau of "fragment" and "amnesia") because the socket buffer forgets that its fragment was shared.

## Why the Corruption Becomes Host Root

The corruption sits in the kernel's global page cache, which is shared between every process on the host regardless of which namespace any of those processes lives in. The attacker performs the corruption from inside a user namespace where they hold namespace-root, but the corrupted bytes are visible to any process that subsequently reads the same file, including privileged processes running outside the namespace.

The privilege escalation completes when a setuid-root binary like `/usr/bin/su` is executed from outside the namespace. The kernel loads the binary from the corrupted page cache, sees the setuid bit on the on-disk file, sets effective UID to 0, and then executes the attacker's shellcode under that effective UID. The shellcode calls `setuid(0)`, which promotes the calling process to real UID 0, and then `execve("/bin/sh")`, which spawns a host-root shell.

This two-stage pattern matters for the practical exploit walkthrough in Task 4. The published proof-of-concept performs only stage one (the corruption) and demonstrates its success by spawning a shell from inside the namespace. That shell appears to be root by `whoami`, but it cannot read root-owned files because it is namespace-root, not host-root. Stage two, which is what produces the usable host-root shell, is triggered by the operator running `/usr/bin/su` manually from outside the namespace after the proof-of-concept exits.




## The 1-Byte Write Primitive

Each AES-GCM decryption produces a one-byte controlled write into the page cache at one chosen offset. The attacker controls the byte by varying the lower 32 bits of the 8-byte IV. Encrypting the counter block under the known AES-128-GCM key yields a 16-byte keystream; only the byte at position 2 byte 0 is used. By iterating the IV nonce through up to 65,536 values, all 256 possible byte values are reachable.

The exploit pre-computes a 256-entry lookup table mapping target bytes to the IV values that produce them. With the table built, each write is a single trigger that completes in microseconds. The public proof-of-concept overwrites the first 192 bytes of `/usr/bin/su` with a small ELF stub that calls `setgid(0); setuid(0); execve("/bin/sh", ...)` at its entry point. The exploit's per-byte loop skips bytes whose current value already matches the desired stub; against `/usr/bin/su` on a typical Linux system, around sixteen bytes coincide (most of the ELF header), so roughly 176 actual triggers run. The full sequence completes in a few hundred milliseconds.

The on-disk file is never touched. File integrity tools that hash from disk continue to report the file as clean. This is identical to the behaviour seen in Copy Fail and Dirty Frag, and the consequences are the same.

# The Patch That Became The Vulnerability

## The Dirty Frag Patch

The original Dirty Frag xfrm-ESP bug ( `CVE-2026-43284` ) was that `esp_input()` took a fast path skipping `skb_cow_data()` for any uncloned, non-linear skb with no frag list. The fast path was unsafe because the skb's fragments could be backed by page-cache pages planted via `splice()` .

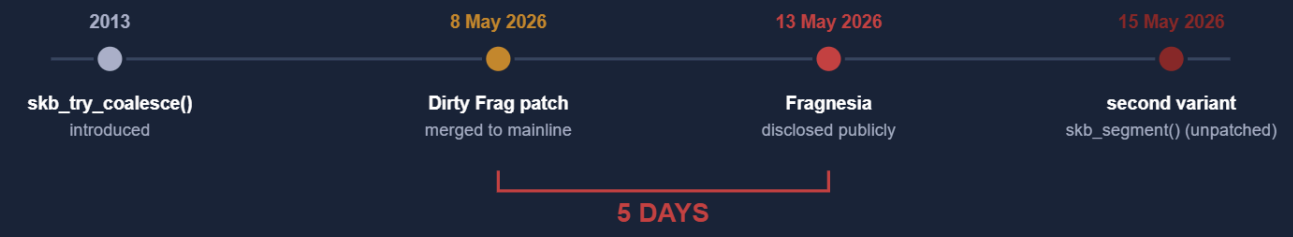
The patch, commit `f4c50a4034e6` , added two pieces of code. First, the IPv4 and IPv6 datagram append paths were updated to set `SKBFL_SHARED_FRAG` on splice'd fragments. Second, `esp_input()` was updated to check `skb_has_shared_frag()` before taking its fast path. The reasoning was correct, the implementation was correct, and the patch closed the original Dirty Frag bug as intended.

What the patch did not do was audit every code path that touched `SKBFL_SHARED_FRAG` to verify it preserved the flag. `skb_try_coalesce()` was one such path. It already existed, had been present since 2013, and had been incorrectly stripping the flag for thirteen years. Until the Dirty Frag patch, no production code path made a critical decision based on the flag, so the bug was latent. After the patch, `esp_input()` started making exactly that decision.

The candidate Fragnesia patch carries two `Fixes:` tags. One points at the Dirty Frag patch ( `f4c50a4034e6` ). The other points at the original `skb_try_coalesce()` commit from 2013 ( `cef401de7be8` ). The tags tell two stories in two lines: the immediate trigger, and the latent precondition.

## The Patch That Became a Vulnerability

How the Dirty Frag fix turned a latent 2013 bug into Fragnesia




### Before the Dirty Frag patch

(2013 through 7 May 2026)

- esp_input() did not check the flag
- skb_try_coalesce() dropped it anyway
- no consumer made a decision on it
- the flag-strip bug was latent

LATENT

### After the Dirty Frag patch

(commit f4c50a4034e6, 8 May 2026)

- esp_input() now checks the flag
- skb_try_coalesce() still drops it
- esp_input() trusts a stale flag
- page-cache writes become reachable

EXPLOITABLE

## The Patch Series

The candidate Fragnesia patch ( `f84eca581739` , on the netdev list since 13 May 2026) is anchored on a two-line change to `skb_try_coalesce()` . When transferring paged fragments between buffers, the patched function now propagates the `SKBFL_SHARED_FRAG` flag from the source skb to the destination skb. The invariant `esp_input()` depends on is restored.

The full series is larger than the headline two-line fix suggests. Beyond the core repair to `skb_try_coalesce()` , it includes a flag-propagation fix in `pskb_copy()` , an XFRM change to avoid in-place decrypt on shared skb fragments, and a cluster of follow-up fixes to rxrpc covering DATA and RESPONSE packet unsharing, re-decryption of RESPONSE packets, rxkad alignment, memory leaks in `rxkad_verify_response()` , and a potential use-after-free. The size of the follow-up series is itself an indicator that the bug class is broader than any single bug.

The implication is broader still. The same audit should be repeated for every function that transfers fragments between skbs, because any function that strips the flag is potentially another Fragnesia waiting for a downstream consumer to make a decision based on it.

## The Second Variant

On 15 May 2026, two days after the initial Fragnesia disclosure, the same V12 researcher published a second variant. This variant bypasses the merged fix by exploiting `skb_segment()` instead of `skb_try_coalesce()` . The function propagates `SKBFL_SHARED_FRAG` only from the head skb when building GSO segments, so a frag_list member carrying page-cache-backed fragments with the flag set will produce segment skbs that have lost the marker.

Triggering the second variant requires three network namespaces connected by veth pairs and is less reliable than the first because it depends on GRO coalescing two segments in the same NAPI poll cycle. As of the room's publication date, the second variant has no merged fix. A separate patch has been proposed on netdev that would incidentally help, but it was not written to address this specific bug.

The operational point is that Fragnesia is unlikely to be the last vulnerability of its shape. The bug class is invariant violations on `SKBFL_SHARED_FRAG` . Every function in the kernel's skb plumbing that transfers fragments is a candidate location for the next instance.

**Info:** The original Dirty Frag bugs took nine years and six years respectively to be found from their introducing commits. Fragnesia took five days to be found from the commit that introduced it. The rate of discovery in this code area has shifted, and the V12 team has publicly attributed part of that to agentic AI-assisted auditing. Whether or not that framing is accurate, the operational reality is that defenders should treat kernel patches in this subsystem as changes to the attack surface, not as completed mitigations.

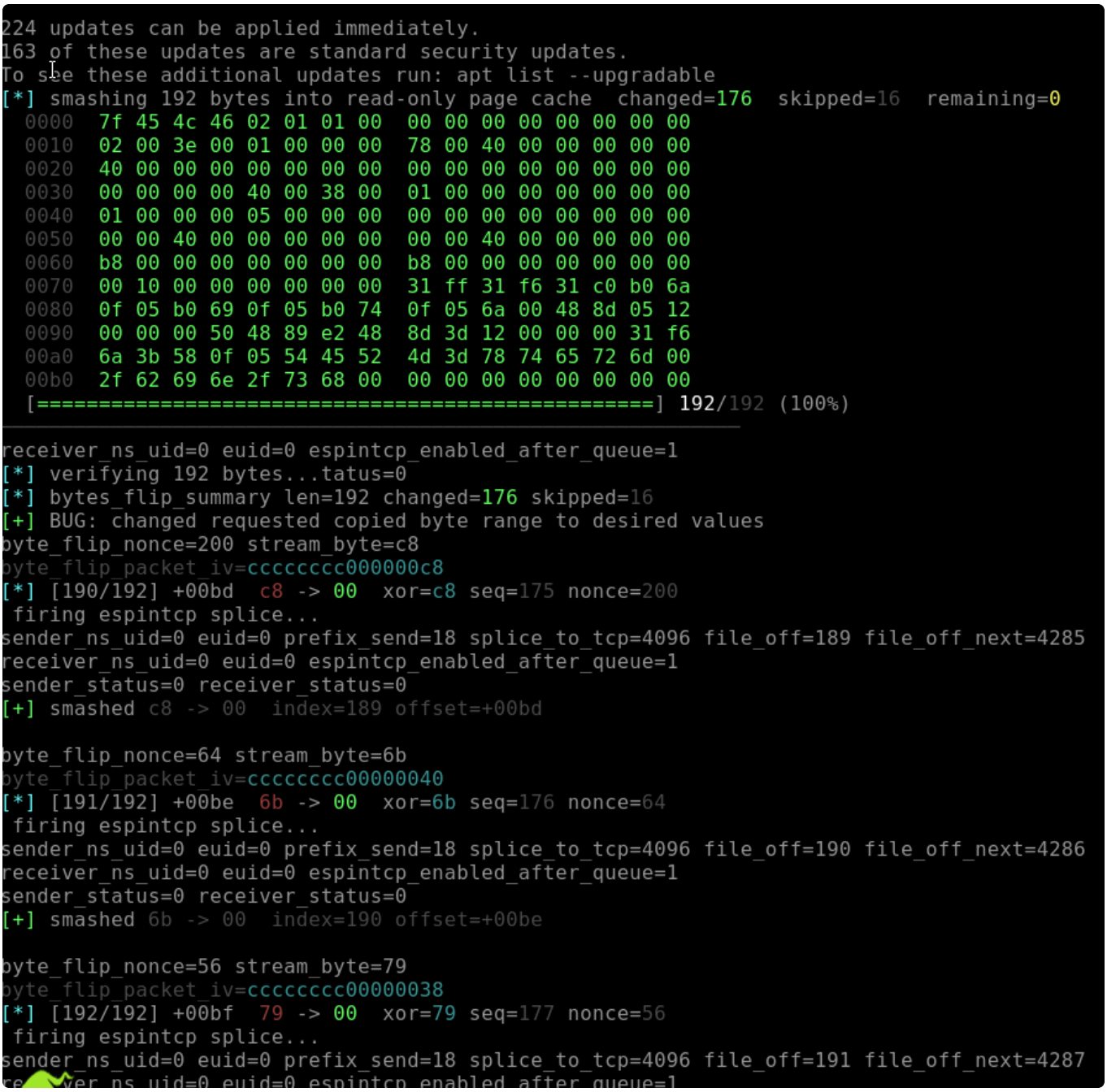
# Exploitation

cd /home/karen/fragnesia gcc -O2 -w fragnesia.c -o exp

the path is where the exploit lives at

I then ran the exploit to corrupt the page cache

by doing ./exp



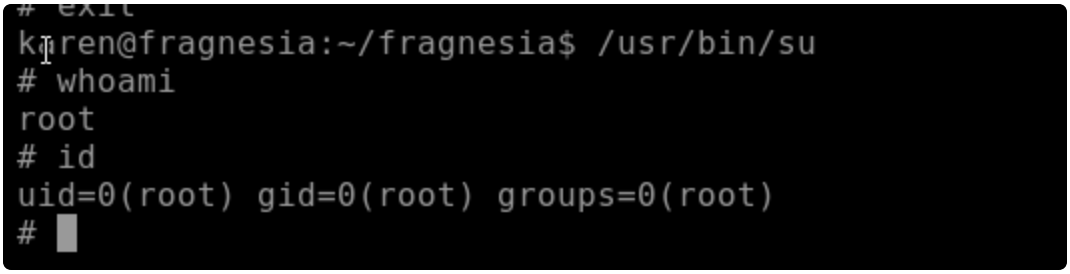

The shell prompt has changed from `$` to `#`, and `whoami` returns `root`. This is the proof-of-concept's own verification step, but the shell it lands in is not yet a host-root shell. A few details from this output deserve closer attention before moving on to stage two.

- The line `outer_write_open_denied=1 errno=13` confirms that karen does not have write access to `/usr/bin/su` from outside the namespace. This is the permission boundary the exploit is about to cross.
- The line `userns_setup: ... ns_uid=0 ns_gid=0` shows that karen is mapped to UID 0 inside a new user namespace, granting `CAP_NET_ADMIN` for the XFRM operations that follow without any real privileges on the host.
- The `changed=176 skipped=16` line is significant. The exploit's ELF stub is 192 bytes long, but `/usr/bin/su` is itself an ELF binary, so its first few bytes ( `7f 45 4c 46 02 01 01 00`

`...`) already match the stub. The per-byte loop skips those, and only the 176 bytes that differ are actually written.

- The shell that appears after `[+] BUG: ...` is the proof-of-concept's `execve("/usr/bin/su")` call running inside the user namespace. Inside the namespace, karen is mapped to UID 0, so the shellcode's `setuid(0)` is a no-op and the resulting shell is namespace-root.
- The `cat /root/flag.txt` returns `Permission denied` because `/root/flag.txt` is owned by the host's real UID 0, which the namespace cannot read. The namespace presents files owned by unmapped UIDs as owned by `nobody` (UID 65534).

I now need to trigger corruption from outside the namespace




1. karen's shell `execve`s `/usr/bin/su`. The on-disk file has the setuid bit set with owner root.
2. The kernel honours the setuid bit and sets the process's effective UID to 0 (real host UID 0, since karen's shell is in the initial user namespace).
3. The kernel loads the binary's bytes from the page cache, which still contains the corrupted ELF stub.
4. The corrupted entry point runs the shellcode under effective UID 0.
5. The shellcode calls `setgid(0)` and `setuid(0)`. Because the effective UID is already 0 at the host level, `setuid(0)` succeeds and sets the real UID to 0 as well.
6. The shellcode calls `execve("/bin/sh", ...)`, spawning a shell with real UID 0.




to verify the on disk file u use the command

sha256sum /usr/bin/su

The hash matches a freshly-installed Ubuntu kernel build. The exploit only modified the in-memory page cache copy; the on-disk binary was never touched. AIDE, Tripwire, IMA, and any other file integrity tool that reads from disk would report this file as clean. The corruption only exists in the kernel's page cache and is visible only to processes that read through that cache.

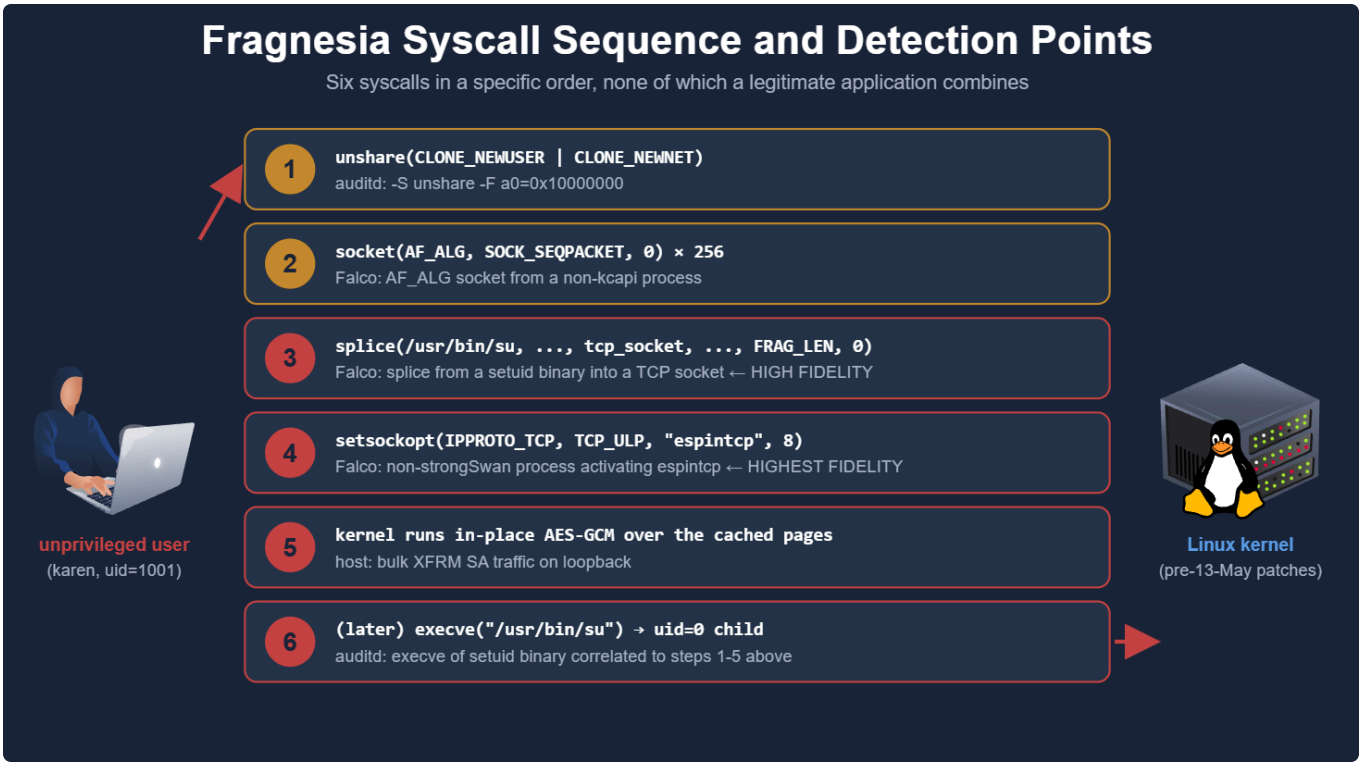
```
# sha256sum /usr/bin/su
5867b33b08c01[illegible]e11f62d325a1625c30a35bea5f29f46babda62aa2cd202e125  /usr/bin/su
# exit
karen@fragnesia:~/fragnesia$ sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

# Detection and Mitigation

## Detection

The exploit's signature is a sequence of system calls that no legitimate application produces:

- `unshare(CLONE_NEWUSER | CLONE_NEWNET)` followed by writes to `/proc/self/uid_map`
- `socket(AF_ALG, ...)` calls used to build the keystream lookup table
- A `socket(AF_INET, SOCK_STREAM)` followed shortly by `setsockopt(SOL_TCP, TCP_ULP, "espintcp")`
- `splice()` from a setuid binary file descriptor into the same TCP socket
- A burst of `XFRM_MSG_NEWSA` netlink messages registering a security association with a known AES-128-GCM key
- An eventual `execve("/usr/bin/su", ...)` and a child process running with effective UID 0




# Key Syscalls to Monitor

| Syscall | What to Watch For | Relevance |
| --- | --- | --- |
| `unshare(CLONE_NEWUSER \| CLONE_NEWNET)` | Unprivileged process creating both namespaces | High-confidence first step |
| `setsockopt(TCP_ULP, "espintcp")` | Process not in known strongSwan/IPsec daemon list | Specific to Fragnesia, very low false positive |
| `socket(AF_ALG, ...)` with `SOCK_SEQPACKET` | Process not in `kcapi-*` or `cryptsetup` set | Same signal used in Copy Fail detection |
| `splice()` from a setuid binary fd to a TCP socket fd | Page-cache page entering the TCP receive queue | Trigger preparation |
| `XFRM_MSG_NEWSA` netlink with known weak key | Unusual SA registration pattern | Bulk SA registration in microseconds |

## MITRE ATT&CK Mapping

| Technique | MITRE ID | Where it appears |
| --- | --- | --- |
| Exploitation for Privilege Escalation | T1068 | The kernel LPE itself |
| Abuse Elevation Control Mechanism: Setuid and Setgid | T1548.001 | `execve("/usr/bin/su")` after page cache corruption |
| Escape to Host | T1611 | When the exploit is run from inside a container |
| Indicator Removal | T1070 | Manual `drop_caches` after exploitation |

## Mitigation

**Step 1: Apply the Module Denylist**

```
sudo sh -c 'printf "install esp4 /bin/false\ninstall esp6 /bin/false\ninstall rxrpc /bin/false\n" > /et
sudo rmmod esp4 esp6 rxrpc 2>/dev/null
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

The configuration file blocks future loads of `esp4`, `esp6`, and `rxrpc`. The `rmmod` line unloads any currently-loaded copies. The `drop_caches` line evicts any corrupted pages already in the cache.

**Step 2: Verify the Block Is Active**

```
sudo modprobe esp4
```

The command should return an error. The module is blocked from loading.

**Step 3: Confirm the PoC Fails**

Re-run the exploit:

```
/home/karen/fragnesia/exp
```

The exploit should fail when the espintcp ULP setsockopt is rejected because the `esp4` module is no longer available. The chain cannot proceed past that point.